

Exercícios para Sala de Aula

16/06/2026 (atualizada) Prof. Wellington Sarmento

1) O Detetive de Divisores

Contexto: Muitos iniciantes criam laços que sobrecarregam o computador sem necessidade. Vamos mostrar como a matemática pode poupar o processador.

O Desafio: Escreva uma função chamada `contarDivisores(n)` que recebe um número inteiro positivo e retorna quantos divisores ele tem.

1. **Primeira Versão da Função:** Na primeira versão de código para responder ao desafio desta questão, use um laço `for` que percorre de 1 até n , como base para seu teste de quais são os divisores do número n .
2. **Primeira Versão da Função:** Como estratégia para uma segunda solução para o desafio desta questão, sugiro que usem a estratégia do *Trial Division Algorithm*, usado para identificar um número **primo**. Nesta abordagem, ao invés de percorrer todos os números entre os valores **1 e n** , o laço vai de **1 até $\sqrt{n}$** . Se encontrarmos um divisor a que seja menor que $\sqrt{n}$, podemos automaticamente descobrir outro divisor, b , fazendo $b = n/a$. Dessa forma, podemos procurar que números entre **1 até $\sqrt{n}$** são divisores de n , caso exista, este número seria o **a** . Calculamos o **b** , através da expressão $b = n/a$. Assim, teremos os **pares divisores** do número n . **Como calcular o número total de divisores na prática, usando este processo descrito?** Ao criar o seu algoritmo com um laço `for()` testando a variável i de 1 até o valor inteiro positivo de $\sqrt{n}$, a lógica de contagem fica assim:

- Se n tiver resto zero ao ser dividido por i , você encontrou um divisor;
- Você então verifica quem é o parceiro dele: $j = \frac{n}{i}$.
- Se $i \neq j$, você soma **2** à sua contador de divisores - pois acabou de encontrar dois divisores distintos -, que você pode chamar de *contDivisores*, por exemplo;
- Se $i = j$ (o que só acontece se n for um quadrado perfeito), você soma apenas **1** ao contador, para não contar o mesmo número duas vezes.

COMO SE CHEGOU NESSA IDEIA? Matematicamente, podemos afirmar que um número inteiro positivo - ou seja, Natural - n é composto pela multiplicação de dois outros números de mesma natureza, menores ou iguais a n , tal que: $n = a \times b$, podendo ser, $a > b$ ou $a < b$ ou $a = b$. No último caso, $a = b$, temos um número **quadrado perfeito** (exemplo: $25 = 5 \times 5$). Podemos afirmar, também, que podemos compor os valores de n a partir de valores pequenos para a e grandes para b . Vejamos um exemplo prático deste processo no quadro a seguir:

Análise de Divisores de $n = 36$ ($\sqrt{36} = 6$) | Teste (p) |

Divide 36?		Cálculo do Par ($q = 36/p$)		Divisores Encontrados	
Observação		:—: :—: :—: :—: :—:		1 Sim $36/1 = 36$	
1 e 36		Primeiro par encontrado (os extremos).		2 Sim	
$36/2 = 18$		2 e 18		Ambos são divisores legítimos.	
$36/3 = 12$		3 e 12		Ambos são divisores legítimos.	
$36/4 = 9$		4 e 9		Ambos são divisores legítimos.	
Nenhum		O resto da divisão não é zero.		6 Sim $36/6 = 6$	
6		$p = q$ (Quadrado perfeito).		Conta-se apenas uma vez.	

Refaça a rotina contarDivisores(n), usando estas informações.

3. Use `console.time()` e `console.timeEnd()` para comparar o tempo de execução das duas versões com o número 999.999.999.

2) Inversão de *Array*

Contexto: A troca de valores (*swap*) usando uma variável temporária é uma das manobras mais fundamentais na programação.

O Desafio: Crie uma função chamada `inverterArray(lista)` que inverta a ordem dos elementos de um array ****sem usar o método nativo `.reverse()`****.

1. Você deve percorrer apenas **metade** do array.
2. Em cada passo, troque o elemento da posição `i` pelo elemento da posição correspondente no final do array.
3. **Regra de Ouro:** Declare a variável `temp` obrigatoriamente dentro do bloco do laço `for`, utilizando `let`.

Exemplo de entrada: [10, 20, 30, 40] **Saída esperada:** [40, 30, 20, 10]

3) O Filtro de Elementos (Iteração de Array com for)

Contexto: Arrays são a base para gerenciar coleções de dados. Antes de utilizarmos métodos avançados, é essencial dominar a iteração manual, pois ela nos dá controle total sobre o fluxo de execução.

O Desafio: Crie uma função chamada `filtrarPares(numeros)` que receba um array de números inteiros.

1. Utilize um laço `for` clássico (com inicialização, condição e incremento) para percorrer cada elemento do array.
2. Identifique os números pares e adicione-os a um novo array chamado `pares`.
3. Retorne o array `pares` ao final da execução.

Exemplo de entrada: [3, 8, 15, 22, 42, 7] **Saída esperada:** [8, 22, 42]

4) O Padronizador de Nomes (Alteração de Strings com `toUpperCase`)

Contexto: Quando recebemos dados de usuários, é comum que a formatação venha bagunçada (como letras maiúsculas no meio de palavras). Padronizar esses dados é uma tarefa diária no desenvolvimento de software.

O Desafio: Crie uma função chamada `formatarNome(nomeBaguncado)` que receba uma string com o nome de uma pessoa escrito de forma irregular.

1. A função deve transformar a primeira letra em maiúscula utilizando `.toUpperCase()`.
2. O restante do nome deve ser transformado em letras minúsculas utilizando `.toLowerCase()`.
3. Concatene as duas partes e retorne o nome corrigido.

Exemplo de entrada: "mArIA" **Saída esperada:** "Maria"

5) O Caçador de Domínios (Extração de Substrings)

Contexto: Frequentemente precisamos extrair apenas um “pedaço” de um texto maior, como a sigla de um estado em um endereço completo ou o domínio de um e-mail para descobrir a empresa do usuário.

O Desafio: Crie uma função chamada `extrairDominio(email)` que receba um endereço de e-mail completo.

1. Utilize o método `.indexOf()` para encontrar a posição do caractere `@`.
2. Use o método `.slice()` ou `.substring()` para extrair apenas o domínio (tudo o que vem logo após o `@`).
3. Retorne o domínio extraído.

Dica: O método `.slice(inicio)` extrai da posição `inicio` até o final da string caso o segundo parâmetro seja omitido.

Exemplo de entrada: "aluno.estudioso@universidade.edu.br" **Saída esperada:** "universidade.edu.br"

6) A Porta de Segurança (Comparação de Strings)

Contexto: O JavaScript é *case-sensitive*, o que significa que “Senha” e “senha” são consideradas palavras diferentes. Além disso, espaços acidentais podem causar falhas em sistemas de login.

O Desafio: Escreva uma função `verificarAcesso(senhaDigitada, senhaCadastrada)` que compara duas strings.

1. A função deve remover quaisquer espaços em branco no início e no final de ambas as strings usando `.trim()`.
2. Em seguida, deve realizar uma comparação **ignorando letras maiúsculas e minúsculas** (transforme ambas para o mesmo caso antes de comparar).

3. Retorne `true` se forem iguais e `false` caso contrário.

Exemplo de entrada: " Secreta123 ", "secreta123" **Saída esperada:**
`true`

7) O Gerador de Crachás (Operações com Strings)

Contexto: Construir mensagens ou documentos dinâmicos juntando textos fixos com variáveis é o que torna as páginas web interativas e personalizadas para cada usuário.

O Desafio: Crie uma função `gerarCracha(nome, departamento, id)` que una essas informações.

1. A função deve concatenar os parâmetros para formar o texto exato do crachá.
2. Você pode utilizar o operador `+` ou as modernas *Template Strings* (crases `e${}`).
3. O formato de retorno deve ser estritamente: `[ID] - NOME: departamento`.

Exemplo de entrada: "Carlos", "TI", 404 **Saída esperada:** "[404] - CARLOS: TI" (Observe que o nome deve ficar todo em maiúsculo na saída!)

8) A Calculadora de Descontos

Contexto: As funções podem receber valores, chamados **parâmetros**, armazená-los em variáveis e efetuar operações sobre estes valores, retornando o resultado deste processamento. Desta forma, elas agem como pequenas “fábricas”, processando entradas e gerando resultados, como saída.

O Desafio: Crie uma função `calcularPrecoFinal(valorProduto, porcentagemDesconto)` que processe uma venda.

1. A função deve calcular quanto vale o desconto em dinheiro.
2. Subtraia esse desconto do valor original do produto.
3. Retorne o valor final a ser pago pelo cliente.

Dica: O cálculo do desconto é feito pela fórmula: $Desconto = Valor \times \frac{Porcentagem}{100}$.

Exemplo de entrada: 150, 20 **Saída esperada:** 120

9) O Mensageiro Poliglota

Contexto: Em sistemas maduros, muitas vezes o usuário não preenche todas as opções disponíveis. Podemos usar “parâmetros padrão” (*default parameters*) para assumir valores automáticos sem precisarmos escrever muitos blocos de `if/else`.

O Desafio: Escreva uma função `saudarUsuario(nome, idioma = "pt")`.

1. A função deve verificar o parâmetro `idioma`.
2. Se for `"pt"`, retorne `"Olá, [nome]!"`. Se for `"en"`, retorne `"Hello, [nome]!"`. Se for `"es"`, retorne `"¡Hola, [nome]!"`.
3. Teste a função omitindo o segundo parâmetro para ver o valor padrão em ação.

Exemplo de entrada: `saudarUsuario("Ana")` **Saída esperada:** `"Olá, Ana!"`

10) Encurtando o Código

Contexto: As *Arrow Functions* surgiram no ES6 para tornar o código mais limpo e conciso, sendo amplamente utilizadas como funções de callback e em métodos de manipulação de arrays modernos.

O Desafio: Observe a função clássica abaixo, que converte uma temperatura de Celsius para Fahrenheit:

```
function converterParaFahrenheit(celsius) {
  return (celsius * 1.8) + 32;
}
```

1. Reescreva essa mesma lógica criando uma **função anônima** e atribuindo-a a uma constante chamada `converterParaFahrenheit`.
2. Utilize a sintaxe de *Arrow Function* (`=>`).
3. Se possível, simplifique-a para o formato de retorno implícito (em uma única linha, omitindo as chaves `{}` e a palavra `return`).

11) A Lista de Compras

Contexto: Enquanto o laço `for` tradicional exige um controle manual de índices, o método `.forEach()` nos permite executar uma função diretamente para cada item de um array, deixando a leitura do código mais natural (declarativa).

O Desafio: Você tem um array de produtos: `let carrinho = ["Maçã", "Pão", "Leite", "Café"]`.

1. Utilize o método `.forEach()` para percorrer o array.
2. Para cada elemento, exiba no console uma mensagem formatada contendo a posição do item e o nome dele.
3. O índice deve começar em 1, não em 0 (apenas visualmente).

Saída esperada no console:

```
Produto 1: Maçã
Produto 2: Pão
Produto 3: Leite
Produto 4: Café
```

PARADA OBRIGATÓRIA: O que significa a sigla ES6?

No **Exercício 10**, mencionamos que as *Arrow Functions* surgiram no **ES6**. Mas o que exatamente é isso? O **ES6** é a abreviação de **ECMAScript 6** (oficialmente chamado de ECMAScript 2015). O ECMAScript é o “manual de regras” ou a especificação padrão que dita como a linguagem JavaScript deve funcionar em qualquer navegador.

A sexta edição desse manual (ECMAScript Version 6 - ES6), lançada no ano de 2015, foi uma das maiores atualizações sofridas pela linguagem JavaScript. Ela modernizou completamente a linguagem e introduziu ferramentas que hoje consideramos indispensáveis no dia a dia, como: * A sintaxe enxuta das **Arrow Functions** (`=>`); * A criação de variáveis com **let** e **const** (que trouxeram mais segurança em relação ao antigo **var**); * As **Template Strings** (o uso de crases e `${}` que vimos no Exercício 7); * Os **Parâmetros Padrão** em funções (que vimos no Exercício 9).

É por causa do impacto gigantesco dessa atualização que, no mundo do desenvolvimento web, frequentemente dividimos a história da linguagem entre o “JavaScript Clássico” (antes de 2015) e o “JavaScript Moderno” (do ES6 em diante).

12) O Inventário de Produtos (Objetos Literais Básicos)

Contexto: Variáveis simples são ótimas para dados isolados, mas na vida real os dados andam agrupados. Um produto de e-commerce, por exemplo, possui nome, preço e quantidade em estoque. Objetos literais nos permitem modelar essas entidades do mundo real encapsulando suas propriedades em uma única estrutura estruturada por chaves (`{}`).

O Desafio: Crie um objeto literal chamado `produto` que represente um item de tecnologia.

1. O objeto deve conter as propriedades: `nome` (string), `preco` (number) e `emEstoque` (boolean).
2. Adicione uma propriedade que seja uma função interna (um método) chamada `calcularTotalEstoque(quantidadeDesejada)`. Esse método deve multiplicar o preço do produto pela quantidade enviada por parâmetro e retornar o valor monetário.
3. Imprima no terminal uma frase utilizando *Template Strings* que acione o método criado.

Exemplo de Objeto Interno: `nome: "Teclado Mecânico", preco: 250`
Saída esperada no console: `"O valor para 3 unidades de Teclado Mecânico é R$ 750"`

13) O Relatório do Pannel (Vetor de Objetos Literais e Iteração)

Contexto: Sistemas de gestão e dashboards não lidam com apenas um objeto, mas sim com listas deles. Em JavaScript, representamos isso combinando vetores (Arrays) com objetos literais. Saber iterar sobre essas estruturas e extrair propriedades específicas é uma habilidade fundamental para manipular dados vindos de APIs e bancos de dados.

O Desafio: Crie um array chamado `usuarios` contendo 3 objetos literais. Cada objeto deve representar um usuário com as propriedades: `id`, `nome` e `ativo` (boolean).

1. Utilize o método `.forEach()` para percorrer o vetor `usuarios`.
2. Para cada usuário cadastrado, verifique se a propriedade `ativo` é verdadeira.
3. Exiba no terminal uma mensagem personalizada apenas para os usuários que estiverem com o status ativo no sistema.

Exemplo de entrada (Array):

```
let usuarios = [  
  { id: 1, nome: "Wellington", ativo: true },  
  { id: 2, nome: "Ana", ativo: false },  
  { id: 3, nome: "Carlos", ativo: true }  
];
```

Saída esperada no console:

```
Usuário Ativo: Wellington (ID: 1)  
Usuário Ativo: Carlos (ID: 3)
```

14) A Média da Turma (Acumulação em Vetores de Objetos)

Contexto: Além de apenas listar ou filtrar dados de coleções de objetos, frequentemente precisamos processar valores numéricos contidos dentro deles, como somar o faturamento de vendas ou calcular médias de notas de alunos em um diário de classe.

O Desafio: Escreva uma função chamada `calcularMediaTurma(listaAlunos)` que receba um array de objetos literais, onde cada objeto possui as propriedades `nome` e `nota`.

1. Utilize um laço de repetição (`for` clássico ou `.forEach()`) para ler a nota de cada objeto dentro do array.
2. Acumule os valores dessas notas em uma variável de soma e calcule a média aritmética simples da turma.
3. Retorne o valor final da média arredondado para uma casa decimal usando o método `.toFixed(1)`.

Exemplo de entrada:

```
let alunos = [  
  { nome: "Tiago", nota: 8.5 },  
  { nome: "Beatriz", nota: 9.0 },  
  { nome: "Lucas", nota: 6.5 }  
];
```

Saída esperada: 8.0

15) A Fábrica de Carros (Criação de Classes e Instanciação)

Contexto: Objetos literais são excelentes para estruturas rápidas, mas se precisarmos criar centenas de objetos com as mesmas propriedades e métodos, copiar e colar chaves se torna inviável. O ES6 introduziu a palavra-chave `class`, que funciona como uma “fôrma” ou “fábrica” de objetos, permitindo a instanciação padronizada de novas entidades através do operador `new`.

O Desafio: Crie uma classe chamada `Carro` que sirva de molde para fabricar automóveis virtuais.

1. A classe deve possuir um método especial chamado `constructor` que receba os parâmetros `marca`, `modelo` e `ano`, atribuindo-os às propriedades internas da instância através do operador contextual `this`.
2. Adicione um método comum chamado `obterDescricao()` dentro da classe que retorne o texto: "Este carro é um [marca] [modelo] ano [ano]".
3. Instancie dois carros diferentes usando o operador `new` e exiba o resultado do método `obterDescricao()` de ambos no console.

Exemplo de chamada: `const meuCarro = new Carro("Toyota", "Corolla", 2024);` **Saída esperada no console:** "Este carro é um Toyota Corolla ano 2024"

16) O Simulador de Conta Bancária (Métodos de Classe com Estado Mutável)

Contexto: Objetos e classes não servem apenas para armazenar dados estáticos; eles gerenciam estados que mudam ao longo do tempo através de interações seguras. Em sistemas bancários ou de jogos, chamamos métodos específicos para alterar valores internos, simulando as regras de negócio reais da aplicação.

O Desafio: Crie uma classe chamada `ContaBancaria` para gerenciar transações financeiras básicas.

1. O `constructor` deve receber o nome do `titular` e iniciar uma propriedade interna chamada `saldo` sempre com o valor padrão de 0.
2. Crie um método chamado `depositar(valor)`. Esse método deve somar o valor recebido por parâmetro diretamente ao `saldo` da conta.

3. Crie um método chamado `sacar(valor)`. Esse método deve verificar se o saldo atual é suficiente para realizar a operação. Se for, subtraia o valor do saldo; caso contrário, exiba no console a mensagem "Saldo insuficiente para o saque de R\$ [valor]".

Exemplo de teste de execução:

```
const minhaConta = new ContaBancaria("Prof. Wellington");
minhaConta.depositar(100);
minhaConta.sacar(150); // Deve exibir aviso de saldo insuficiente
```

Referências para Estudo

Abaixo, uma lista de referências para leitura que ajudarão na resolução dos exercícios.

- **Estruturas de repetição (Laço for):** MDN - for statement
- **Eloquent Javascript:** Capítulo 03
- **Manipulação e Métodos de String:** MDN - String
- **Extração de textos (slice vs substring):** MDN - String.prototype.substring()
- **Template Literals (Strings complexas):** MDN - Template strings
- **Funções e Passagem de Parâmetros:** MDN - Functions
- **Parâmetros Predefinidos (Default Parameters):** MDN - Parâmetros Predefinidos
- **Arrow Functions:** MDN - Arrow function expressions
- **Iteração em Arrays (forEach):** MDN - Array.prototype.forEach()
- **Trabalhando com Objetos (MDN):** MDN - Working with Objects
- **Classes em JavaScript (MDN):** MDN - Classes